6.Implementation of a Simple Artificial Neural Network (ANN) Using the Iris Dataset

What is Artificial Neural Network (ANN)?

An Artificial Neural Network (ANN) is a machine learning model inspired by the biological structure of the human brain. It consists of interconnected processing units called neurons that work together to recognize patterns and make predictions.

ANNs are widely used for:

Image Classification Speech Recognition Medical Diagnosis Recommendation Systems Stock Price Prediction Fraud Detection

Structure of ANN

A basic ANN consists of three layers:

1. Input Layer

Receives the input features from the dataset.

For the Iris dataset:

Sepal Length Sepal Width Petal Length Petal Width

Total Input Neurons = 4

2. Hidden Layer

Processes the information received from the input layer.

Each neuron performs:

Output=Activation(Weight×Input+Bias)

Common activation functions:

ReLU Sigmoid Tanh

3. Output Layer

Produces the final prediction.

Since Iris has 3 classes, output neurons = 3

Iris-setosa Iris-versicolor Iris-virginica

Softmax activation is generally used for multiclass classification.

In [1]:
```
# Import Libraries
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

In [2]:
```
# Load Dataset

data = pd.read_csv(r"C:\Users\Smit\OneDrive\Desktop\Dataset\IRIS.csv")
print(data.head())
```

```
   sepal_length  sepal_width  petal_length  petal_width      species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa
```

In [3]:
```
# Features and Target

X = data.iloc[:, :-1]
y = data.iloc[:, -1]
```

In [4]:
```
# Encode Target

encoder = LabelEncoder()
y = encoder.fit_transform(y)
```

In [5]:
```
# Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

In [6]:
```
# Feature Scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [7]:
```
# Build ANN Model

model = Sequential()
model.add(Dense(8, activation='relu', input_shape=(4,)))
model.add(Dense(8, activation='relu'))
model.add(Dense(3, activation='softmax'))
```

```
C:\Users\Smit\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:8
7: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a lay
er. When using Sequential models, prefer using an `Input(shape)` object as
the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [8]:
```
# Compile Model

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

In [9]:
```
# Train Model

model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=5,
    verbose=1
)
```

```
Epoch 62/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 779us/step - accuracy: 0.9923 - loss: 0.0
858
Epoch 63/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 938us/step - accuracy: 0.9777 - loss: 0.0
803
Epoch 64/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 902us/step - accuracy: 0.9806 - loss: 0.0
895
Epoch 65/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9767 - loss: 0.072
2
Epoch 66/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 812us/step - accuracy: 0.9785 - loss: 0.0
883
Epoch 67/100
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 894us/step - accuracy: 0.9677 - loss: 0.0
762
Epoch 68/100
```

In [10]:
```
# Evaluate Model

loss, accuracy = model.evaluate(X_test, y_test)
print("\nAccuracy :", accuracy)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step - accuracy: 1.0000 - loss: 0.0457

Accuracy : 1.0
```

In [11]:
```
# Predict

predictions = model.predict(X_test)
predicted_classes = np.argmax(predictions, axis=1)

print("\nPredicted Classes")
print(predicted_classes)

print("\nActual Classes")
print(y_test)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step

Predicted Classes
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]

Actual Classes
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```